FORM-TL-01
Notas de presentación: Vecindario

MACIAS VASQUEZ CESAR ABRAHAN

Periodo académico 2026–2027

1. Operaciones principales del diagrama

1.1. Mensajería directa, presencia y moderación

Las cifras de esta sección (500 ms, 1 min, 99,5 %) son *objetivos de diseño*; se comprobarán con pruebas en PE-U1.

Recorrido de un mensaje directo. Al abrir la aplicación, el cliente (CLI) establece un canal WebSocket seguro (WSS sobre TCP) con el balanceador, que lo asigna a una de las instancias de MSG (S4). El canal queda abierto en ambos sentidos. Un mensaje pasa por tres tramos con sincronía distinta:

1. **Envío y ACK de guardado (síncrono para el remitente).** El cliente genera un `message_id` único y envía el mensaje; MSG lo guarda en PostgreSQL (S6) y solo entonces responde al remitente con el **ACK de guardado**. El remitente espera ese ACK y, si no llega, reenvía con el mismo `message_id`.

2. **Entrega al destinatario (asíncrona).** Después de guardar, MSG empuja el mensaje por el canal WSS del destinatario si está conectado. El ACK al remitente no espera a esta entrega.

3. **ACK de entrega (asíncrono).** El cliente del destinatario confirma la recepción con el `sequence_no` recibido; MSG registra ese valor como el último confirmado para ese destinatario en esa conversación.

Por tanto, el **ACK al remitente significa "guardado en PostgreSQL", no "entregado"**. La entrega se confirma aparte con el ACK de entrega del destinatario.

Presencia en línea. Mientras el canal está abierto, el cliente envía periódicamente un *heartbeat* por el mismo canal WSS (S5); no es una petición que bloquee al cliente, por lo que se trata como asíncrono y periódico. Con cada uno, MSG renueva en Redis la clave de presencia del usuario, que tiene un tiempo de expiración (TTL) (S7). Si los *heartbeats* dejan de llegar, la clave expira y el usuario se muestra como desconectado, sin necesidad de un aviso explícito.

Orden: asignación de `sequence_no` entre varias instancias de MSG. Como hay varias instancias, el número de secuencia no puede asignarse en memoria. Se asigna en PostgreSQL, que es el punto común: cada conversación tiene un contador y, dentro de una misma transacción, MSG incrementa ese contador y guarda el mensaje con el valor obtenido. El bloqueo de la fila del contador hace que dos instancias que reciben mensajes de la misma conversación al mismo tiempo se ordenen una detrás de otra y no puedan obtener el mismo número. La tabla de mensajes impone que el par (`conversation_id`, `sequence_no`) y el `message_id` sean únicos.

Deduplicación y cursor de recuperación. Antes de asignar un número, MSG busca el `message_id`: si ya existe (por ejemplo, un reintento tras perder el ACK), devuelve el `sequence_no` ya asignado y no crea otro mensaje ni avanza el contador. Para recuperarse, el cliente conserva por conversación el último `sequence_no` confirmado y, al reconectarse (F1 o F3), lo envía como cursor; MSG responde con los mensajes de esa conversación cuyo `sequence_no` es mayor, en orden. Si el cliente detecta un salto en la numeración, pide el rango que falta.

Denuncia y ocultamiento. Cuando un usuario denuncia un aviso, el cliente envía a MOD el `id_aviso`, el `id_usuario` y el motivo (S8), y MOD confirma la recepción. MOD cuenta únicamente las denuncias válidas de usuarios distintos. Al llegar a tres, solicita a PUB ocultar el aviso (S9); PUB cambia su estado a oculto, que es la fuente de verdad, y publica en Kafka el evento `PublicaciónOcultada` (A4), que MUR consume para retirar o invalidar el aviso en los muros.

ID	**Origen → destino**	**Mensaje**	**Sincronía**	**Protocolo y middleware**
S4 (envío)	CLI remitente → balanceador → MSG	Mensaje y ACK de guardado	Síncrona para el remitente	WSS/TCP
S4 (entrega)	MSG → CLI destinatario	Entrega y ACK de entrega	Asíncrona	WSS/TCP
S5	CLI ↔ MSG	*Heartbeat*	Asíncrona, periódica	WSS/TCP
S6	MSG ↔ PostgreSQL	Guardar mensaje y contador	Síncrona (antes del ACK)	Protocolo PostgreSQL/TCP
S7	MSG ↔ Redis	Presencia con TTL	Síncrona breve	RESP/TCP
S8	CLI → MOD	Denuncia y confirmación	Síncrona	HTTPS/TCP, REST API
S9	MOD → PUB	Solicitud de ocultar	Síncrona	HTTP/2, gRPC
A4	PUB → Kafka → MUR	`PublicaciónOcultada`	Asíncrona	TCP, Kafka

Mecanismos y justificación.

- **WSS sobre TCP, bidireccional y persistente (S4, S5).** Un canal abierto permite empujar el mensaje al destinatario apenas se guarda, sin abrir una conexión por cada envío, y sirve también para el *heartbeat*. TCP garantiza entrega ordenada y confiable; WSS cifra el tráfico.

- **PostgreSQL síncrono antes del ACK (S6).** Un mensaje no se confirma al remitente hasta estar guardado, para no confirmar algo que luego pueda perderse.
- **Redis con TTL (S7).** La presencia es un dato temporal que cambia constantemente y no necesita sobrevivir a un reinicio, por lo que se guarda en memoria con expiración.
- **REST y gRPC en la moderación (S8, S9) y Kafka en la invalidación (A4).** La denuncia necesita confirmación inmediata; la orden de ocultar es una llamada directa entre servicios internos; y el retiro del aviso en los muros puede completarse de forma asíncrona.

R2: objetivo de 500 ms y orden. El canal WSS ya está abierto, por lo que la entrega a un destinatario conectado no depende de establecer conexiones nuevas y la tarea más costosa de la ruta es una escritura en PostgreSQL. El `sequence_no` asignado en PostgreSQL mantiene el orden por conversación aunque haya varias instancias, y el `message_id` único evita duplicados. Se plantea como objetivo y se medirá en PE-U1.

R5: objetivo de ocultar en menos de un minuto. La solicitud a PUB (S9) es síncrona y cambia el estado de inmediato. El evento A4 es asíncrono, así que por sí solo no garantiza el límite: si el evento aún no se consumió, MUR podría seguir mostrando su copia del aviso. Para cerrar ese hueco, MUR no se fía solo del evento: al armar el muro para un usuario, consulta a PUB en lote el estado de los avisos que va a mostrar, con una caché de pocos segundos, y descarta los ocultos. Así, tras el cambio de estado en PUB, el aviso deja de verse en cuanto expira esa caché, y el evento A4 se encarga después de limpiar los muros ya armados. Se plantea como objetivo y se medirá en PE-U1.

R6: objetivo de disponibilidad de 99,5 % en la mensajería. Se aspira a esa meta con varias instancias de MSG detrás del balanceador, *health checks* que retiran las instancias que no responden y persistencia de los mensajes en PostgreSQL. El muro está desacoplado mediante Kafka, de modo que su degradación no bloquea la mensajería.

F1: el destinatario está desconectado. El mensaje ya se guardó en PostgreSQL antes del ACK de guardado, por lo que queda pendiente aunque el destinatario no esté en línea. Al reconectarse, el cliente envía su cursor (último `sequence_no` confirmado por conversación) y MSG le entrega los mensajes posteriores.

F3: MSG se reinicia con conversaciones abiertas. Los clientes se reconectan al balanceador, que los asigna a una instancia disponible, y recuperan desde su cursor, sin depender de memoria volátil. Los mensajes que el remitente no vio confirmados se reenvían con el mismo `message_id`, y la deduplicación evita que se guarden dos veces.